Cooperative Kernel regression

Mahdi HADJ TAIEB, Rayen MANSOUR, Rayen ZARGUI

Mentors: Andrea SIMONETTO - Romain PUJOL

Context

As a non-parametric machine learning approach, kernel ridge regression is designed to estimate a continuous function based on noisy data points. By utilizing kernel functions, it overcomes the limitations of standard linear regression, allowing for the capture of highly nonlinear relationships between inputs and targets.

Consider a set of n observations denoted by $\{(x_i, y_i)\}_{i=1}^n$. Each label $y_i \in \mathbb{R}$ corresponds to an evaluation of an underlying, unknown function $f : \mathbb{R} \to \mathbb{R}$ at feature $x_i \in \mathbb{R}$, corrupted by an additive Gaussian noise:

$$y_i = f(x_i) + \epsilon_i, \quad \text{with } \epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

The function f is modeled as a linear combination of kernel evaluations:

$$f(x) = \sum_{i=1}^{n} \alpha_i k(x, x_i).$$

Here, $k(\cdot, \cdot)$ represents a symmetric positive definite kernel. For this study, we focus on the Gaussian (or squared exponential) kernel, defined as:

$$k(x, x_i) = \exp(-\|x - x_i\|^2).$$

Given the high computational cost associated with large values of n, we apply a Nyström approximation. This consists of uniformly sampling a much smaller subset of indices $\mathcal{M} \subset \{1, \dots, n\}$ of size m (where $m \ll n$), which simplifies the function's expression to:

$$f(x) \approx \sum_{j \in \mathcal{M}} \alpha_j k(x, x_j).$$

We define $K_{nm} \in \mathbb{R}^{n \times m}$ as the cross-kernel matrix whose elements are $k(x_i, x_j)$ for $i \in \{1, \dots, n\}$ and $j \in \mathcal{M}$. Similarly, let $K_{mm} \in \mathbb{R}^{m \times m}$ denote the kernel matrix computed exclusively on the subset $\mathcal{M}$. The optimal weights α^* are then found by minimizing a regularized empirical risk:

$$\alpha^* = \arg\min_{\alpha \in \mathbb{R}^m} \left(\frac{\sigma^2}{2} \alpha^\top K_{mm} \alpha + \frac{1}{2} \|y - K_{nm}\alpha\|_2^2 + \frac{\nu}{2} \|\alpha\|_2^2 \right).$$

The scalar $\nu > 0$ acts as a regularization parameter, guaranteeing that the objective function is strongly convex and that the numerical resolution remains stable.

This mathematical framework effectively scales up kernel regression for massive datasets without sacrificing theoretical soundness. Building upon this centralized model, the subsequent sections will detail how to adapt and solve this optimization problem cooperatively across a decentralized network of communicating agents.

1 Part I

1.1 Distributed Formulation

We divide the dataset of $n = 100$ samples evenly across $a = 5$ distinct agents. Consequently, each agent i stores a local subset of $n_a = 20$ observations, while the number of selected kernel centers remains $m = 10$. Rather than centralizing the computation, the agents optimize a decentralized formulation of the problem. They iteratively exchange information over a communication graph to reach an agreement on the global minimizer α^*, formulated as:

$$\alpha^* = \arg\min_{\alpha \in \mathbb{R}^m} \sum_{i=1}^{a} \left[\frac{\sigma^2}{2a} \alpha^\top K_{mm} \alpha + \frac{1}{2} \|y_i - K_{(i)m}\alpha\|_2^2 + \frac{\nu}{2a} \|\alpha\|_2^2 \right].$$

To simulate the network topology, we investigate several undirected and connected graph configurations, specifically focusing on line, small-world, and fully connected (complete) graphs.

Let $f_i(\alpha)$ represent the specific loss function assigned to the i-th agent:

$$f_i(\alpha) = \frac{\sigma^2}{2a} \alpha^\top K_{mm} \alpha + \frac{1}{2} \|y_i - K_{(i)m}\alpha\|_2^2 + \frac{\nu}{2a} \|\alpha\|_2^2,$$

where the parameter vector is $\alpha \in \mathbb{R}^m$, and $K_{(i)m}$ stands for the rows of the cross-kernel matrix associated with the data points held by agent i. The overall optimization problem is thus the sum of these local functions:

$$f(\alpha) = \sum_{i=1}^{a} f_i(\alpha).$$

For optimization purposes, the first and second-order derivatives (gradient and Hessian) of the local functions f_i are derived analytically:

$$\nabla f_i(\alpha) = \frac{\sigma^2}{a} K_{mm}\alpha - K_{(i)m}^\top \left(y_i - K_{(i)m}\alpha\right) + \frac{\nu}{a}\alpha,$$

$$\mathcal{H} f_i = \frac{\sigma^2}{a} K_{mm} + K_{(i)m}^\top K_{(i)m} + \frac{\nu}{a} I_m.$$

The global cost function f is guaranteed to be L-smooth and μ-strongly convex, with the constants defined by:

$$L = \max_{1 \le i \le a} \{L_i\}, \quad \mu = \min_{1 \le i \le a} \{\mu_i\},$$

where L_i and μ_i correspond, respectively, to the maximum and minimum eigenvalues of the local Hessian matrix $\mathcal{H} f_i$.

1.2 Dataset

Our dataset consists of one million points (x_i, y_i), where $x_i \in \mathbb{R}$ is the feature of point i, and $y_i \in \mathbb{R}$ is its corresponding noisy label. The goal is to learn a function $f : \mathbb{R} \to \mathbb{R}$ such that $y_i \approx f(x_i)$.

Here is a quick visualisation of the data:

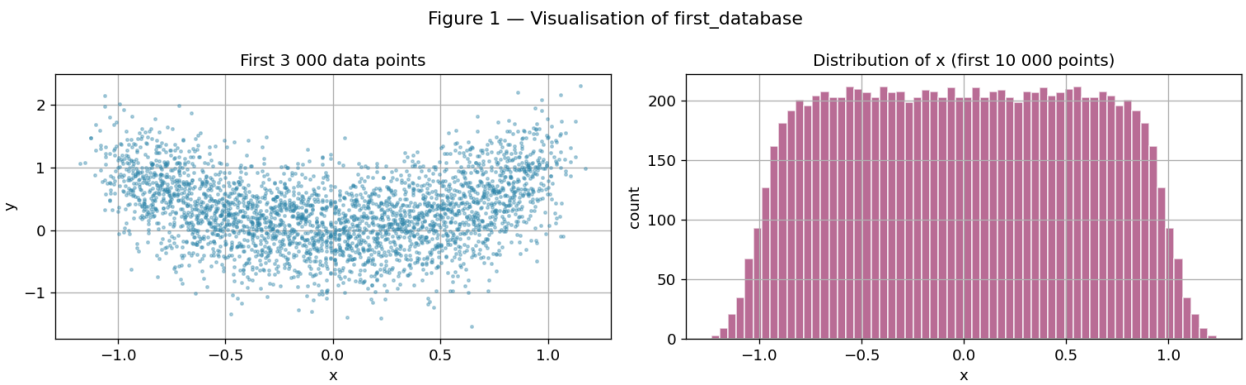


Figure 1: The data points (x_i, y_i)

1.3 Methods

We implemented four distributed optimization algorithms:

- Decentralized Gradient Descent (DGD)
- Gradient Tracking (DGT)
- Dual Decomposition (DD)
- Alternating Direction Method of Multipliers (ADMM)

Each algorithm was tested with different graph structures, and the results were compared in terms of convergence and function approximation.

1.4 Step Size Selection and Theoretical Justification

The convergence of decentralized algorithms is highly sensitive to the choice of the step size η (or the penalty parameter β). Following the theoretical framework of the course, we calibrate these parameters based on the properties of the local cost functions f_i and the network topology.

- **DGD and DGT:** For these first-order methods, the step size η must satisfy the condition $\eta < 2/L$ (or more conservatively $\eta < 1/L$) to ensure stability, where L is the Lipschitz constant of the gradient ∇f_i. In our implementation, we chose $\eta = 0.002$. While DGD with a constant step size only converges to a neighborhood of the optimum due to consensus bias, DGT (Gradient Tracking) utilizes this same step size to achieve exact linear convergence by tracking the global gradient direction.

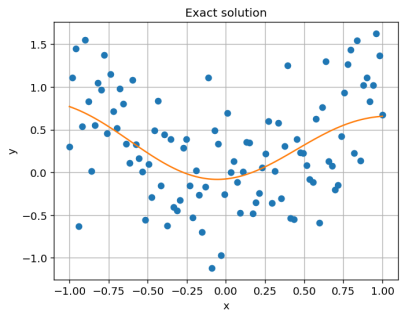
- **Dual Decomposition (DD):** The stability of the dual ascent depends on the strong convexity μ of the primal problem and the spectral properties of the edge-incidence matrix A. Theoretically, the dual step size α should satisfy $\alpha \leq 2\mu/\|\mathcal{A}\|^2$. We set the dual step size to 0.1 and the inner primal learning rate to 10^{-3} with 50 sub-iterations to ensure that the primal sub-problem is sufficiently solved at each step, maintaining the validity of the dual update.

- **ADMM:** Unlike gradient-based methods, ADMM is theoretically guaranteed to converge for any penalty parameter $\beta > 0$ in the convex case. However, β controls the trade-off between primal and dual convergence. We selected $\beta = 100$ to prioritize fast consensus (agreement between agents) across the network. The inner primal updates were performed with a step size scaled by $1/\beta$ to maintain stability within the augmented Lagrangian framework.

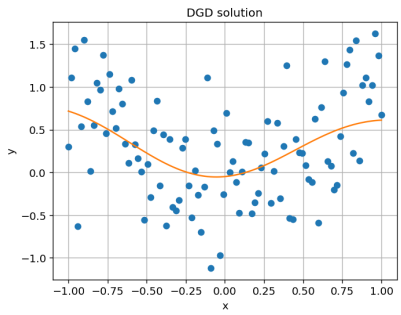
1.5 Function approximation

We fix the graph structure to a line graph for the visual comparison of function approximations in Figure 2, while the influence of the graph topology is studied separately in Subsection 1.7.

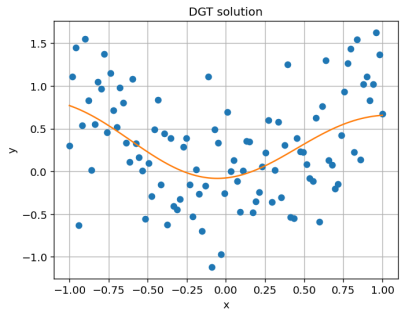
Figure 2 shows the regression curves obtained using the exact centralized solution and the distributed methods.



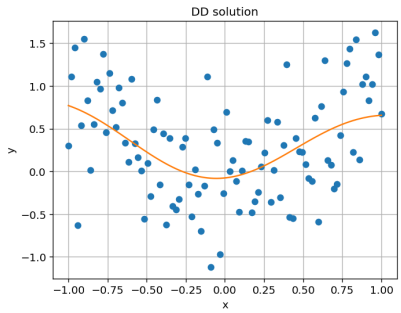
(a) Exact solution



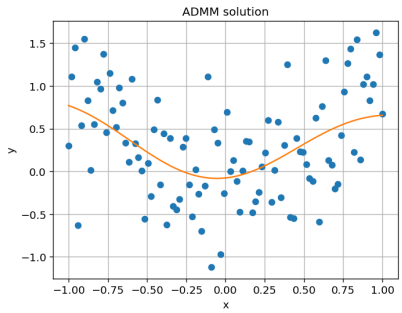
(b) DGD



(c) DGT



(d) DD



(e) ADMM

Figure 2: Function approximations obtained with exact solution and distributed methods.

1.6 Convergence analysis

We evaluate the convergence of the algorithms by plotting the evolution of the optimality gap $\|\alpha_t^i - \alpha^*\|$ over iterations in a log-log scale. Figure 3 compares the convergence of all methods on a fixed graph.

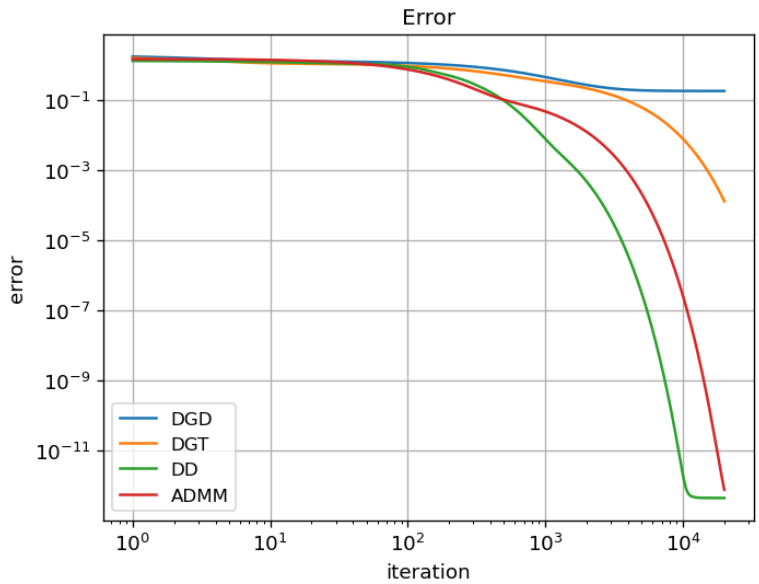


Figure 3: Optimality gap vs iterations (log-log scale) for each method.

We observe that the gradient tracking (DGT) method achieves exact convergence, unlike DGD which stagnates early due to the consensus bias inherent in constant step-size decentralized methods. While both dual-based approaches and DGT reach high precision, Dual Decomposition (DD) reaches machine precision the fastest, significantly outperforming ADMM and DGT in terms of convergence speed for this specific setting. These results are consistent with the theoretical properties studied in class, illustrating how tracking and dual-based mechanisms effectively eliminate consensus error to reach the exact global optimum.

To compute the reference solution α^* for the optimality gap, we solve the centralized problem:

$$\alpha^* = \left(\sigma^2 K_{mm} + K_{nm}^\top K_{nm} + \nu I\right)^{-1} K_{nm}^\top y,$$

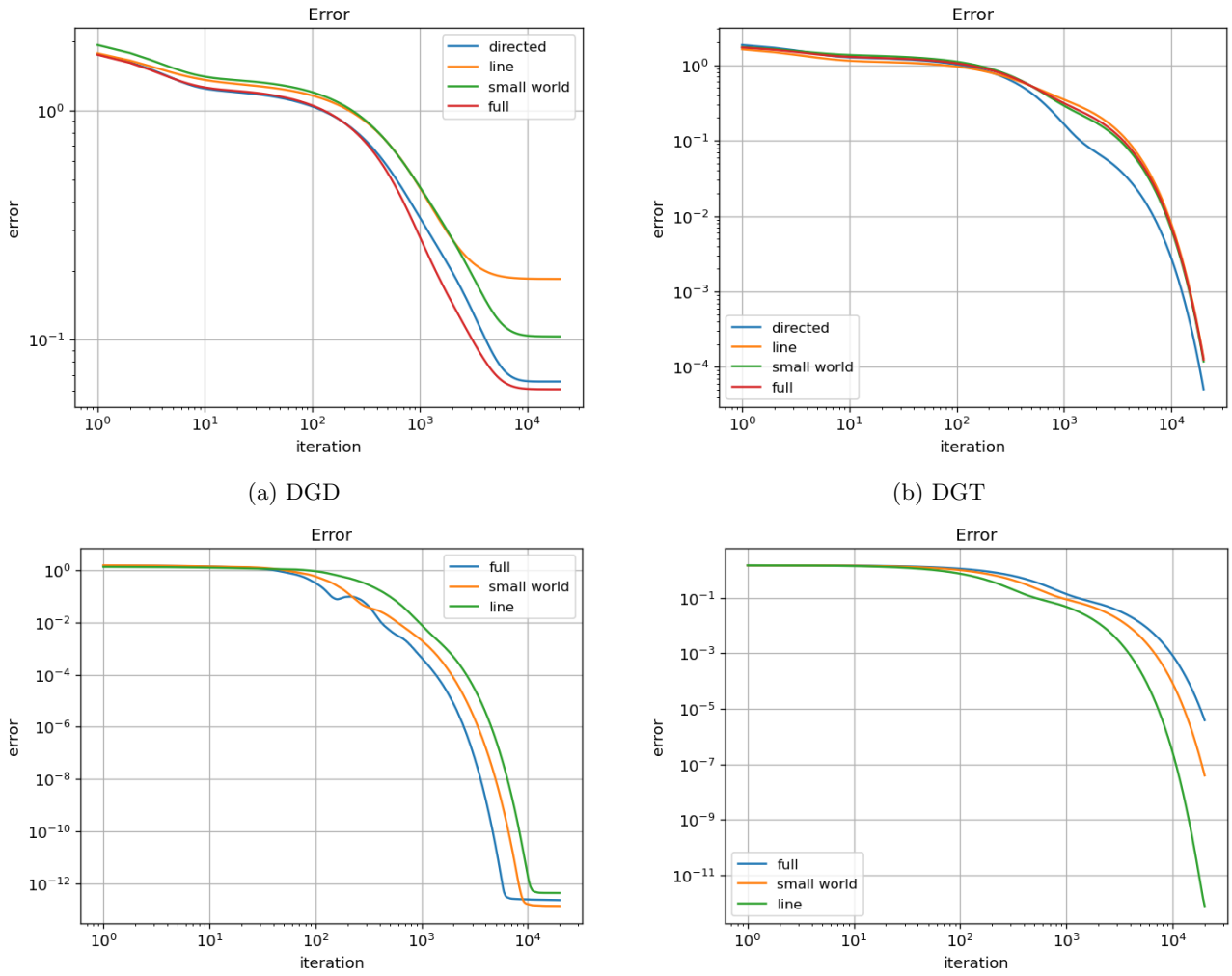
which corresponds to solving a regularized linear system.

1.7 Influence of the Communication Graph Topology

In this section, we analyze how the network structure impacts the convergence speed and stability of the decentralized algorithms. We consider four distinct topologies for a network of $a = 5$ agents:

1. **Line Graph:** A structure with minimal connectivity where information must propagate sequentially through the network.
2. **Small World Graph:** A cycle-based structure with additional "shortcuts" to reduce the network diameter, represented below:
3. **Fully Connected Graph:** A topology where every agent communicates with all others, providing maximal information mixing.
4. **Directed Communication Graph:** A more complex structure where links are not necessarily symmetric:

Figures 4a, 4b, 4c, and 4d illustrate the optimality gap evolution for each topology.



(a) DGD

(b) DGT

(c) DD

(d) ADMM

Figure 4: Optimality gap vs iterations (log-log scale) for different communication graphs, for each method.

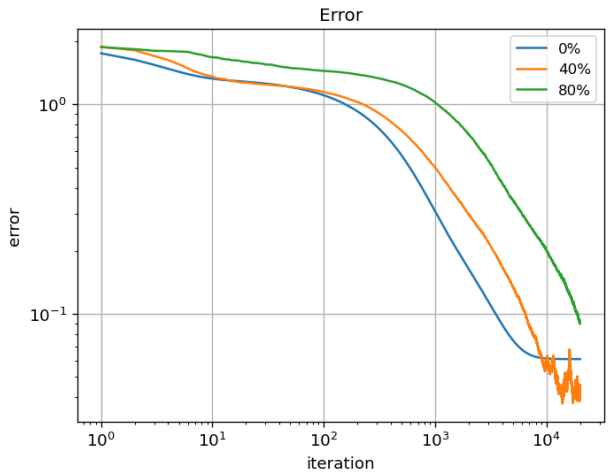
We observe that for all methods, the communication graph significantly impacts the convergence rate. Fully connected graphs consistently yield the fastest convergence, followed by small-world and line graphs. Directed communication leads to instability or divergence due to the lack of symmetry and averaging. This aligns with the theoretical expectations: better connectivity improves information diffusion across agents, accelerating consensus and optimization.

1.8 Robustness to communication failures

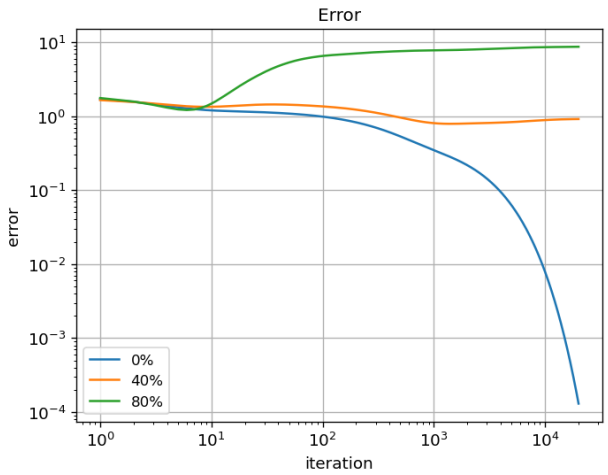
To assess the robustness of distributed methods to imperfect communication, we introduce two types of perturbations during training:

- **Package loss**: agents randomly fail to transmit their updates with a given probability.
- **Asynchronous updates**: agents perform local updates at slightly desynchronized time steps.

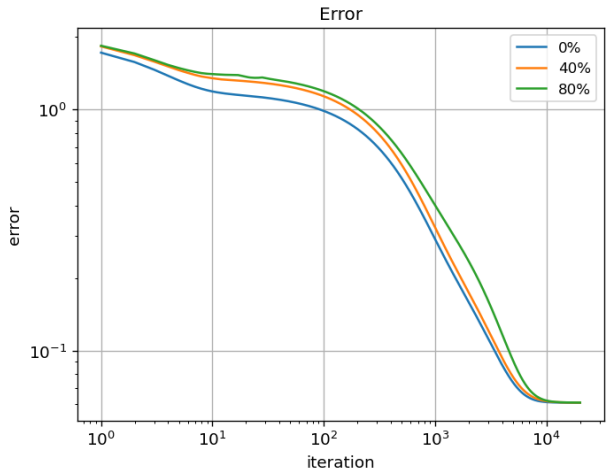
We apply these perturbations to DGD and DGT over a fully connected graph, and compare the resulting optimality gaps.



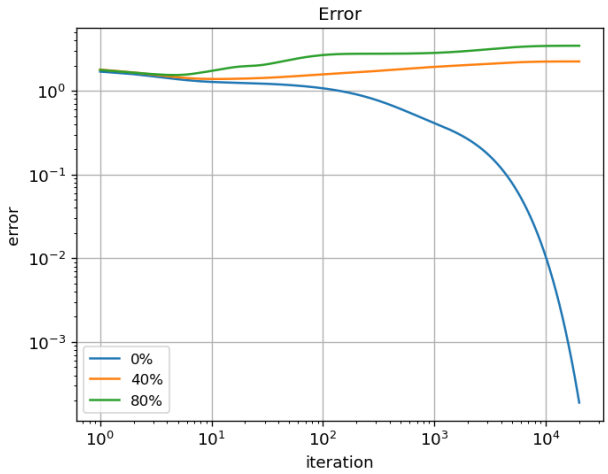
(a) DGD with package loss



(b) DGT with package loss



(c) DGD with asynchrony



(d) DGT with asynchrony

Figure 5: Impact of communication failures on DGD and DGT.

We observe that:

- **Packet Loss:** Communication failures significantly degrade the convergence rate, particularly for DGT. Unlike DGD, which degrades gracefully by settling at a higher error floor, DGT relies on a recursive tracking variable that requires consistent updates to maintain an accurate estimate of the global gradient. Our results show that with a packet loss of 40% or higher, DGT fails to reach machine precision and instead plateaus at an error level comparable to, or worse than, DGD. This is because lost updates break the "conservation" of the gradient tracker, leading to a permanent bias in the descent direction.

- **Asynchronicity:** This factor disrupts the synchronization required for the gradient tracking mechanism. While DGD is relatively robust to asynchronicity (as it only relies on neighborhood averaging) DGT is highly sensitive to the "stale" gradients introduced by asynchronous updates. Even at a 40% asynchronicity rate, the algorithm loses its exact convergence property. The tracking error accumulates over time, preventing the optimality gap from decreasing below a certain threshold (10^{-1} in our experiments), whereas in ideal conditions, it would reach 10^{-15}.

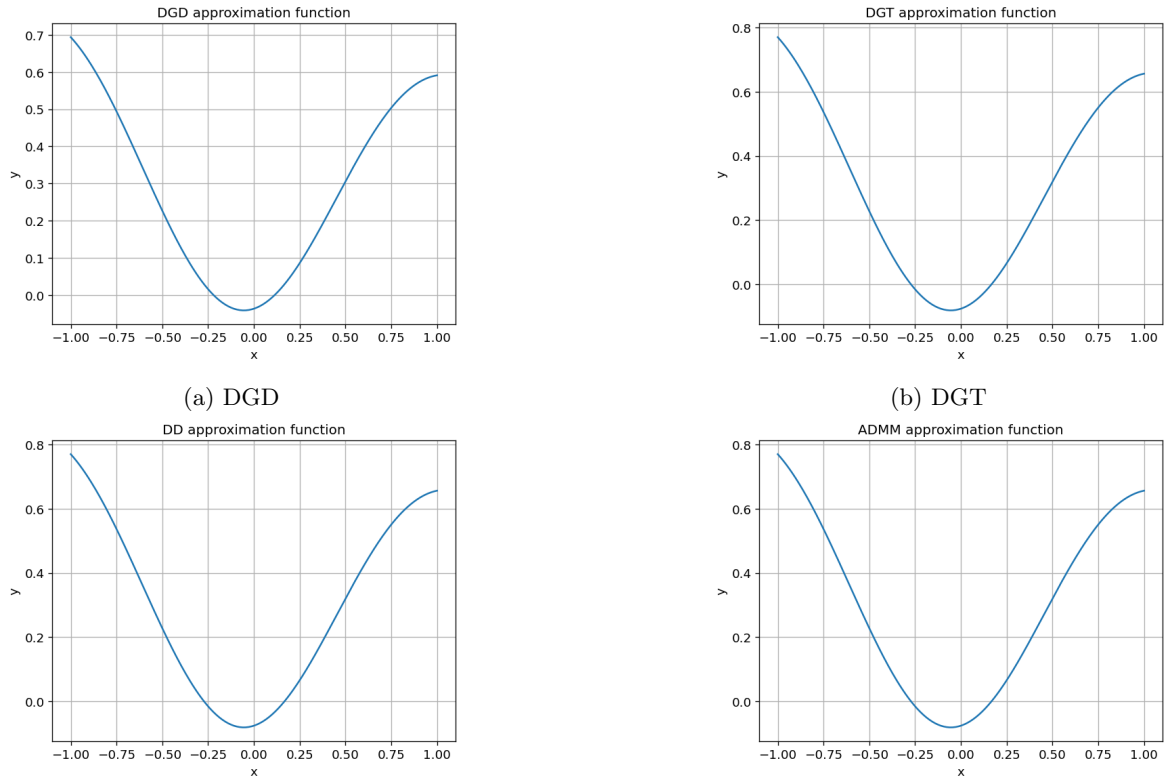
Overall, DGD proves to be more resilient to communication unreliability, whereas DGT, despite its superior speed in ideal settings, is far more fragile. These observations align with the theoretical requirement of DGT for exact and timely consensus to cancel the variance of local gradients.

1.9 Function approximation on a test grid

To visualize the final learned functions, we evaluate them on a uniform test grid of $nt = 250$ points in $[-1, 1]$, defined by:

```
x_prime = np.linspace(-1, 1, nt)
```

Each method's resulting function is then plotted, as shown in Figure 6, using the learned parameters α.



(a) DGD (b) DGT

(c) DD (d) ADMM

Figure 6: Function approximations on a dense evaluation grid ($nt = 250$) for each method.

We observe that all methods are able to recover a similar smooth functional shape, despite differences in their optimization trajectories. The final approximated functions are consistent across algorithms, confirming that each method is effectively minimizing the same objective. The minor variations in amplitude or curvature are likely due to the convergence precision or sensitivity to communication topology. Notably, DGT, DD, and ADMM tend to produce slightly sharper reconstructions compared to DGD.

1.10 (Optional) Push-Sum DGD: Correction for Directed Graphs

Here we address the case where communication is asymmetric (directed graph). Under this configuration, standard DGD fails to reach the true global optimum because it cannot correctly balance information across the agents.

We implemented the **Push-Sum protocol**, which introduces a weight-tracking system to compensate for this imbalance, and compared it against a "Naïve" DGD approach.

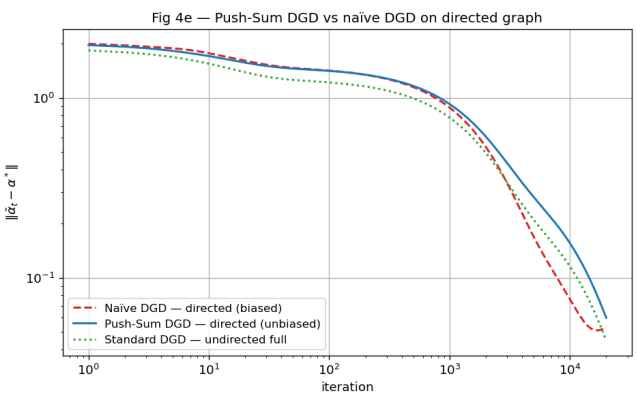


Figure 7: Comparison between Naïve DGD and Push-Sum DGD on a directed graph.

The analysis of Figure 7 reveals:

- **Naïve DGD (Red dashed):** The error plateaus at a high level. Because the graph is directed and the mixing is not doubly stochastic, the algorithm converges to a biased solution where certain agents "count" more than others in the network average.
- **Push-Sum DGD (Blue solid):** The algorithm successfully removes this bias. The curve matches the performance of the standard DGD on undirected graphs (Green dotted), proving that the weight-based normalization effectively recovers the true global optimum.

Conclusion: The Push-Sum protocol is essential for non-symmetric networks. It allows the system to recover the exact solution without changing the physical graph structure, simply by adding a scalar normalization step.

1.11 The impact of the total number of points n

In the previous experiments, we limited ourselves to $n = 100$ points out of the full dataset, which contains one million points. To better assess the scalability and behavior of the algorithms, we now increase the problem size to $n = 500$, while setting $m = 23$ and $a = 50$.

Increasing n helps simulate more realistic and complex scenarios, but it comes with a trade-off: computing the exact optimal solution α^* becomes computationally infeasible. Consequently, we can no longer evaluate the optimality gap, and instead we use the value of the objective function as our error metric.

Figure 8 presents the convergence behavior of the four methods in this larger setting.

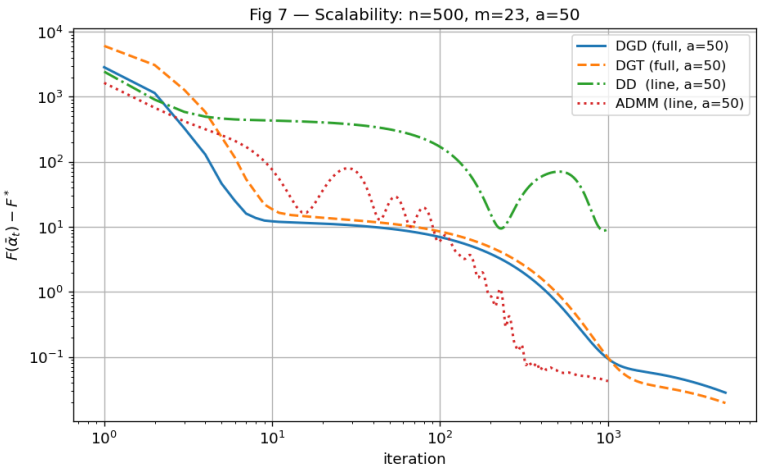


Figure 8: Convergence of the four algorithms with $n = 500$. The error represents the value of the objective function.

At this scale, the decentralized algorithms demonstrate strong consistency by converging toward a similar objective value. While DGD, DGT, and ADMM successfully reach the vicinity of the global minimum, the Dual Decomposition (DD) method exhibits significant instability in this 50-agent network, failing to settle at the optimal value. This highlights that while most methods scale well, maintaining consensus in larger networks remains a challenge that requires more careful hyperparameter tuning.

2 Part II — Federated Learning on the Second Database

We return to the setting $n = 100$, $m = 10$, and $a = 5$, and consider the second database, where the data are already split across the five clients:

```
with open('second_database.pkl','rb') as f:    X,Y = pickle.load(f).
```

By construction, $X[i]$ and $Y[i]$ correspond to the 20 samples available to client i. In this part, the Nyström landmarks do not need to coincide with labeled samples; we therefore use the fixed grid

$$x_m = \texttt{np.linspace}(-1, 1, m),$$

which gives $m = 10$ uniformly spaced landmarks on $[-1, 1]$.

For each client $i \in \{1, \dots, C\}$, with $C = 5$, we define the local objective

$$F_i(\alpha) = \frac{\sigma^2}{2C}\alpha^\top K_{mm}\alpha + \frac{1}{2}\|y_i - K_i\alpha\|_2^2 + \frac{\nu}{2C}\|\alpha\|_2^2,$$

and the global federated objective

$$F(\alpha) = \frac{1}{C}\sum_{i=1}^{C} F_i(\alpha).$$

Throughout this section, we report the *objective optimality gap*

$$\mathcal{E}_t = \left|F(\alpha^t) - F(\alpha^\star)\right|,$$

where $\alpha^\star$ denotes the centralized optimum of the same kernel regression problem.

2.1 Client data distribution

Before running the federated methods, we first inspect how the second database is distributed across the five clients. Figure 9 shows the local datasets. The samples are not perfectly identical across clients, although they still overlap significantly. This indicates a moderately heterogeneous setting, where local training may induce client drift but not as severely as in an extreme non-IID partition.

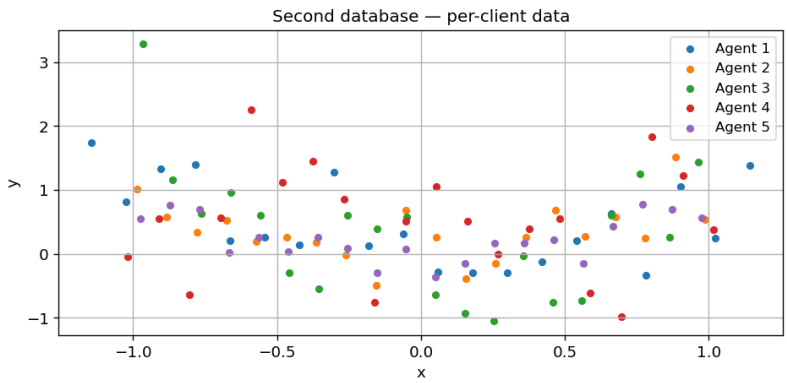


Figure 9: Second database split across the five clients. Each color corresponds to one agent.

2.2 FedAvg experiments

FedAvg alternates between local optimization and server aggregation. At each communication round, the server broadcasts the current global model to the selected clients. Each selected client then performs E local gradient steps (or stochastic gradient steps if $B < 20$) before the updated local models are averaged by the server. Since each client has the same number of samples here, the aggregation reduces to a simple average.

2.2.1 Full-batch setting with constant learning rate: $B = 20$, $S = 5$, $E \in \{1, 5, 50\}$

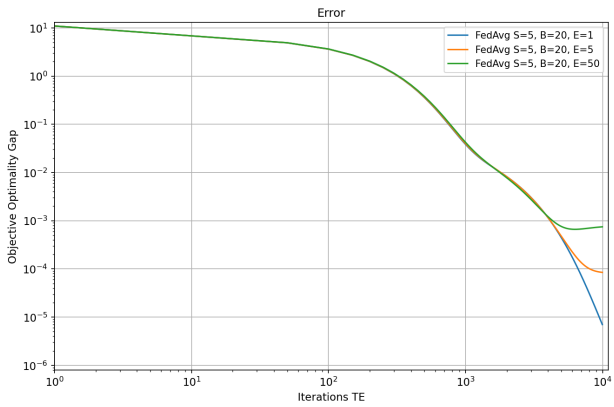
We first consider the setting explicitly requested in the project: all clients participate at every round ($S = 5$), each client uses its full local dataset ($B = 20$), and the local number of epochs is varied in

$$E \in \{1, 5, 50\}.$$

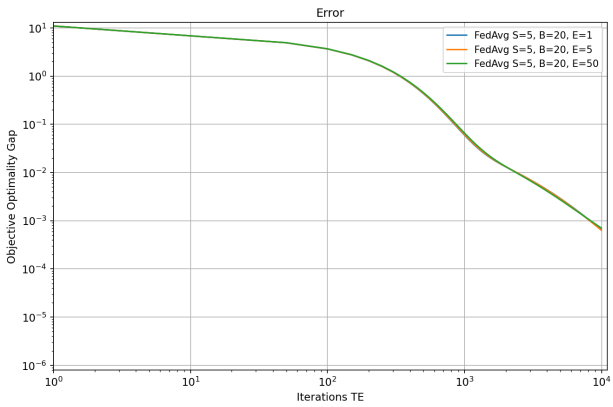
The corresponding convergence curves are shown in Figure 10(a). All three configurations converge, but the final optimization accuracy clearly depends on the number of local epochs. The case $E = 1$ achieves the smallest final error, while the case $E = 50$ reaches a noticeably larger plateau. This is consistent with the standard interpretation of FedAvg: when E is small, the local models remain close to the global one, so the averaged direction remains well aligned with the descent direction of the global objective. By contrast, when E is large, each client moves further toward its own local minimizer before synchronization. If the local objectives are not exactly identical, this produces the classical *client drift* effect, which degrades the final convergence accuracy.

2.2.2 Full-batch setting with diminishing learning rate

We then keep the same deterministic setting but replace the constant learning rate by a diminishing one. Figure 10(b) shows that this schedule makes the three trajectories much closer to each other. The convergence becomes more stable and less sensitive to the choice of E. This is expected: a decreasing learning rate reduces oscillations in the late phase and mitigates the accumulation of local bias. The price to pay is that the method becomes more conservative, but it is also more robust.



(a) FedAvg with constant learning rate, $B = 20$, $S = 5$, $E \in \{1, 5, 50\}$.



(b) FedAvg with diminishing learning rate, $B = 20$, $S = 5$, $E \in \{1, 5, 50\}$.

Figure 10: FedAvg in the full-batch regime. The constant learning rate yields a clearer separation between the different values of E, whereas the diminishing schedule stabilizes the trajectories and reduces the sensitivity to the number of local epochs.

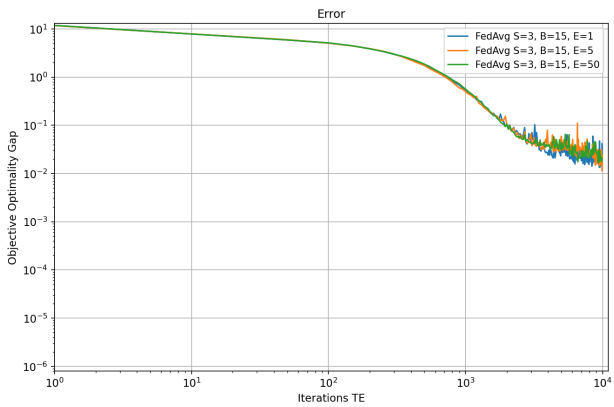
2.2.3 Effect of the batch size: $B = 15$

We next reduce the local batch size from 20 to 15. In this case, gradients are computed on randomly selected subsets of the local data, which introduces stochasticity into the local updates.

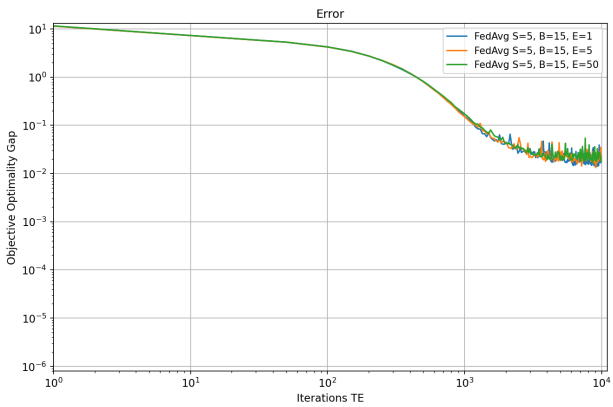
Figures 11(a) and 11(b) display the results for $S = 3$ and $S = 5$, respectively. In both cases, the trajectories become noisier and all curves plateau at a significantly larger error than in the full-batch setting. Moreover, the differences between $E = 1$, $E = 5$, and $E = 50$ become much smaller. This suggests that once stochastic gradient noise dominates, the impact of the number of local epochs is less pronounced than in the deterministic regime. Increasing E no longer changes the qualitative behavior as much as before because the optimization is already limited by sampling noise.

2.2.4 Effect of partial participation: $S = 3$ versus $S = 5$

Comparing Figures 11(a) and 11(b), we observe that reducing the number of active clients from $S = 5$ to $S = 3$ introduces additional randomness, but the overall behavior remains similar. This is natural: partial participation adds variance to the server update, yet the dominant source of error in this setting is already the mini-batch stochasticity induced by $B = 15$. Therefore, the effect of selecting fewer clients is visible but moderate.



(a) FedAvg with $B = 15$, $S = 3$, $E \in \{1, 5, 50\}$.



(b) FedAvg with $B = 15$, $S = 5$, $E \in \{1, 5, 50\}$.

Figure 11: FedAvg with reduced mini-batch size. Compared with the full-batch regime, the trajectories are noisier and the final error floor is much larger.

2.3 SCAFFOLD

SCAFFOLD introduces control variates that correct the mismatch between local gradients and the true global gradient. The local update can be written as

$$\alpha_i^{k+1} = \alpha_i^k - \eta\big(\nabla F_i(\alpha_i^k; \mathcal{B}_i^k) - c_i + c\big),$$

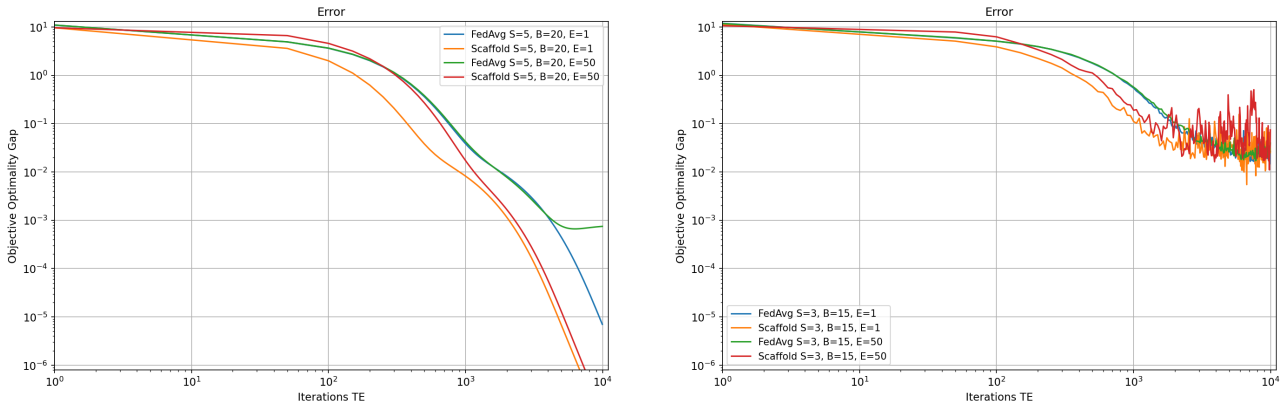
where c_i is the local control variate and c the global one. The purpose of this correction is to reduce client drift, especially in heterogeneous federated settings.

2.3.1 FedAvg vs. SCAFFOLD in the full-batch regime

Figure 12(a) compares FedAvg and SCAFFOLD for $B = 20$, $S = 5$, and two representative values of the local workload, namely $E = 1$ and $E = 50$. The advantage of SCAFFOLD is clear. For $E = 1$, both methods converge well, but SCAFFOLD is slightly faster and reaches a lower error. For $E = 50$, the gain is much more pronounced: while FedAvg quickly plateaus, SCAFFOLD continues to decrease and eventually reaches errors below 10^{-6}. This is exactly the regime where control variates are useful: when the number of local updates is large, client drift becomes the main source of degradation for FedAvg, and SCAFFOLD compensates for it.

2.3.2 FedAvg vs. SCAFFOLD in a more stochastic regime

Figure 12(b) shows the comparison for $B = 15$ and $S = 3$. In this case, the advantage of SCAFFOLD is much smaller. The trajectories of both methods are noisy and remain of the same order of magnitude. This is consistent with theory: SCAFFOLD corrects the *bias* induced by heterogeneity, but it does not eliminate the *variance* introduced by small mini-batches and random client participation. Once stochastic noise dominates, the correction mechanism becomes less decisive.



(a) FedAvg vs. SCAFFOLD with $B = 20$, $S = 5$.

(b) FedAvg vs. SCAFFOLD with $B = 15$, $S = 3$.

Figure 12: Comparison between FedAvg and SCAFFOLD. SCAFFOLD clearly improves the deterministic heterogeneous regime, whereas its advantage becomes much smaller in the more stochastic setting.

2.3.3 Extreme non-IID scenario

Finally, we consider an extreme non-IID partition obtained by sorting the first 100 samples according to their input value and assigning consecutive blocks of 20 samples to the five clients. This creates a strongly heterogeneous federated setting, in which client drift becomes the dominant difficulty.

Figure 13 reports the results for $B = 20$, $S = 5$, and $E \in \{10, 50, 100\}$. The difference between FedAvg and SCAFFOLD is now very clear. FedAvg deteriorates significantly as the number of local epochs increases: for $E = 10$ it still decreases to a moderate error level, but for $E = 50$ and especially $E = 100$ it plateaus at much larger values. By contrast, SCAFFOLD remains stable and consistently outperforms FedAvg for all three values of E.

For $E = 10$, the gain is already visible, while for $E = 50$ and $E = 100$ the improvement becomes substantial: SCAFFOLD continues decreasing toward very small errors, whereas FedAvg is strongly limited by client drift. This matches the theoretical motivation of SCAFFOLD: the control variates compensate for the bias between local and global descent directions, which becomes particularly important in highly heterogeneous settings and for large local workloads.

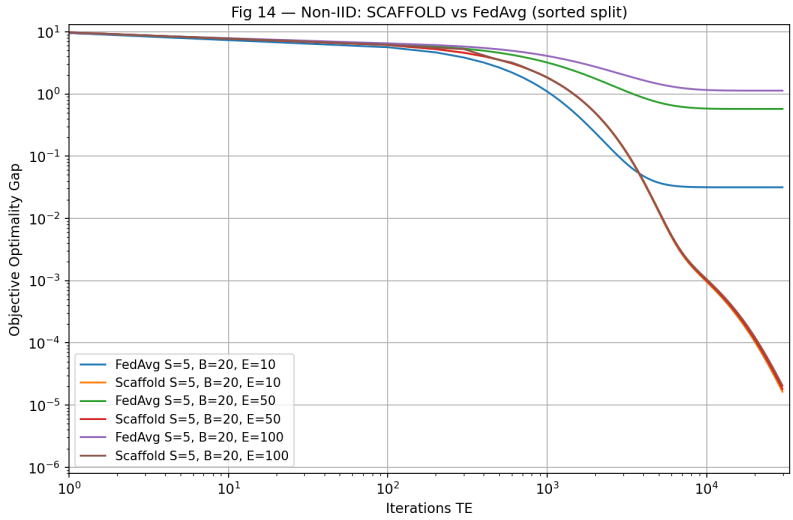


Figure 13: Extreme non-IID experiment with $B = 20$, $S = 5$, and $E \in \{10, 50, 100\}$. In this highly heterogeneous setting, SCAFFOLD remains stable and clearly outperforms FedAvg, especially for large numbers of local epochs.

3 Part III

We go back to the dataset considered in Part I. We now aim to solve the regression problem in a distributed and differentially private (DP) manner. To this end, we employ the DGD-DP algorithm.

We present below the results obtained for $n = 100$:

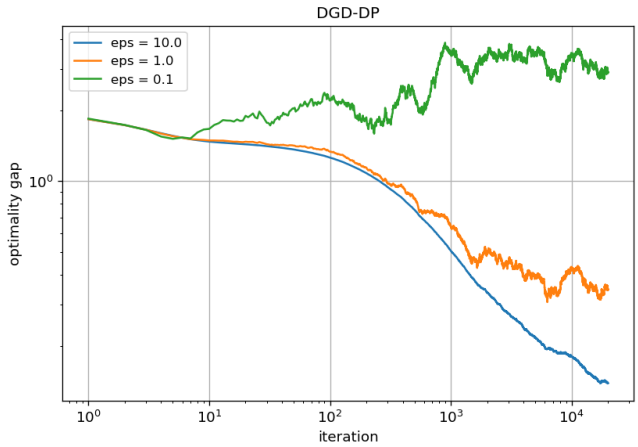


Figure 14: DGD-DP: Optimality gap vs iterations for various levels of privacy ε.

As illustrated in Figure 14, the case $\epsilon = 10$ (low privacy) behaves similarly to the standard DGD algorithm, following a consistent descent before reaching a slightly higher steady-state error of approximately 1.4×10^{-1}. However, as we reduce the privacy budget ϵ, we observe a severe deterioration in convergence performance. This is expected, as stronger privacy guarantees require adding Gaussian noise with higher variance to the gradients, which directly obscures the optimization signal shared across agents.

This highlights a fundamental **privacy-utility trade-off**:

- **Accuracy loss:** For $\epsilon = 1$, the noise floor rises significantly, preventing the algorithm from reaching the precision of the non-private baseline.
- **Convergence failure:** In the high-privacy regime ($\epsilon = 0.1$), the noise power effectively "drowns out" the gradient signal. Instead of descending, the optimality gap increases and fluctuates around 3.0, indicating that the updates have become dominated by a random walk.

Another important observation is the massive increase in **variance** for lower ϵ values. The high-frequency fluctuations visible in the red curve ($\epsilon = 0.1$) are intuitive: noisier updates generate inaccurate local compu-

tations, preventing the network from reaching a stable consensus and causing global instability in the system behavior.

We now consider a larger setting with $n = 1000$ data points, $m = 33$ kernel centers, and $a = 100$ agents. The results are shown below:

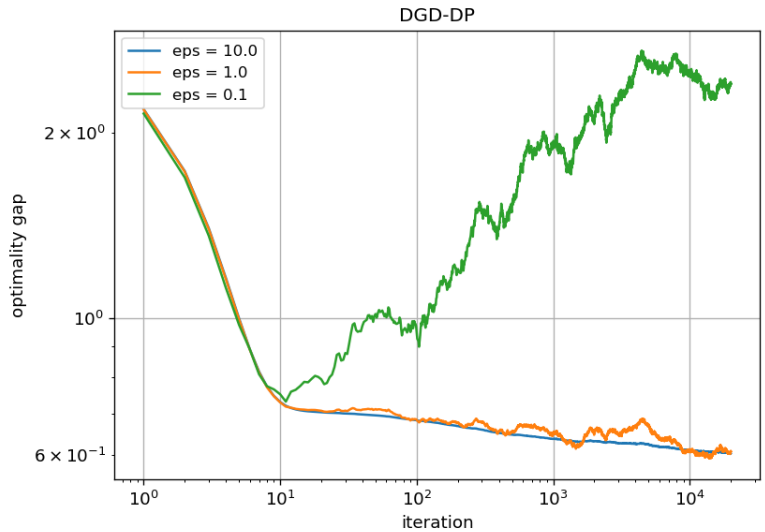


Figure 15: DGD-DP with $n = 1000$, $m = 33$, $a = 100$ for different privacy levels ε.

As illustrated in Figure 15, the scalability test on a larger network of $a = 100$ agents confirms that the privacy-utility trade-off persists as the system grows. In the weak privacy regime ($\epsilon = 10$), the convergence is nearly indistinguishable from the standard DGD baseline, reaching a comparable steady-state error.

At the other extreme, the high-privacy requirement ($\epsilon = 0.1$) leads to a complete failure of convergence; the optimality gap remains high and exhibits heavy oscillations, as the optimization signal is effectively drowned out by the excessive Gaussian noise. The intermediate case ($\epsilon = 1.0$) achieves convergence but settles at a much higher error floor than the baseline. These results highlight that while a larger number of agents might theoretically help in averaging out local noise, the "privacy price" for low ϵ remains the dominant factor, significantly limiting the achievable precision of the decentralized model.

4 Conclusion

This project evaluated distributed kernel ridge regression using DGD, DGT, DD, and ADMM. By leveraging Nyström approximation, we achieved scalability, while DGT and ADMM demonstrated superior robustness and precision compared to DGD. We observed that denser graph topologies significantly accelerate convergence, whereas directed communication and asynchrony pose major challenges, necessitating corrections like the Push-Sum protocol.

In federated settings, Scaffold outperformed FedAvg by mitigating client heterogeneity through its variance-reduction mechanism. Finally, our implementation of DGD-DP confirmed the fundamental privacy-utility trade-off: while high ϵ values recover standard performance, strict privacy ($\epsilon = 0.1$) masks the optimization signal with noise, preventing convergence. Ultimately, the success of decentralized learning relies on balancing algorithmic choice with network topology, communication efficiency, and data protection requirements.